

Backlog arquitectura y trazabilidad de LOGISTPULSE

Integrantes: Martín Endara, Antonio Muñoz y Marco Bonilla

Alcance de la entrega

Organizamos el módulo de pedidos de LOGISTPULSE con diez historias de usuario, un backlog priorizado y estimado, un sprint de cuatro historias, el diseño MVC y una matriz de trazabilidad. El objetivo es registrar un pedido y comprobar el avance del mismo identificador hasta READY.

La base es LOGISTPULSE-GOLDEN del laboratorio. Se conserva su aplicación FastAPI, PostgreSQL y Redpanda; nuestro aporte añade documentación y una prueba de dominio. Los hallazgos LP del PDF inicial vinculan cada historia con la inyección y la elicitación ya desarrolladas.

Contenido

Apartado	Página
Product Backlog	2
Plan de Sprint 1	3
MVC y justificación	4
Matriz de trazabilidad	5
Repositorio README y PR	6
Guía para explicar	7

Cierre de evidencias

Evidencia técnica cerrada: Docker Compose, salud, creación, consulta, secuencia por eventos y persistencia aprobaron sobre el commit identificado. El PR #1 está abierto; la revisión de Antonio o Marco y la fusión siguen pendientes y no se presentan como realizadas.

LOGISTPULSE Product Backlog

INC-LP-01: inepción 2.1 del PDF inicial. TEC-LP-01/02/03: análisis documental, escenarios y prototipado de sus secciones 2.2/2.3/2.4. Los hallazgos LP mantienen los IDs del documento del equipo.

HU / prioridad / puntos	Historia de usuario	Criterio de aceptación	Hallazgo	Previas	Estado
HU-LP-01 Must · 2 pt	Como operador, quiero crear un pedido, para iniciar su preparación.	POST /api/fulfillment/orders con storeId, channel y total: 201, orderId y estado inicial WAITING.	LP-01 / LP-02	—	PASS verificado en Docker
HU-LP-02 Must · 1 pt	Como operador, quiero consultar pedidos recientes, para identificar el que registré.	GET /api/fulfillment/orders: 200; máximo 20 pedidos, del más reciente al más antiguo; contiene el orderId creado.	LP-03	01	PASS verificado en Docker
HU-LP-03 Must · 2 pt	Como responsable de cocina, quiero recibir pedidos nuevos, para iniciar su procesamiento.	Con broker y worker disponibles, ORDER_CREATED se publica en logistpulse.orders y el worker actualiza el mismo orderId a PREPARING.	LP-04	01	PASS verificado en Docker
HU-LP-04 Must · 2 pt	Como operador, quiero observar el avance del pedido, para saber cuándo está listo.	En laboratorio sin cola previa, el mismo id pasa WAITING → PREPARING → READY en ≤ 60 s; cada consulta refleja su estado guardado.	LP-05	02,03	PASS verificado en Docker
HU-LP-05 Should · 1 pt	Como operador, quiero rechazar totales negativos, para evitar pedidos incorrectos.	Total negativo: respuesta 422 o 400 explicando el error y ningún pedido nuevo.	LP-08	01	Pendiente / fuera del sprint
HU-LP-06 Should · 1 pt	Como operador, quiero una confirmación con id, para reconocer el pedido creado.	Después de crear, la pantalla muestra el orderId y una confirmación del estado inicial.	LP-07	01	Pendiente / fuera del sprint
HU-LP-07 Should · 1 pt	Como operador, quiero etiquetas claras de estado, para entender el avance sin depender del color.	WAITING, PREPARING y READY se presentan como En espera, En preparación y Listo, siempre con texto.	LP-09 / LP-15	02	Pendiente / fuera del sprint
HU-LP-08 Should · 2 pt	Como operador, quiero actualizar el listado y reconocer errores, para consultar información vigente.	Actualizar muestra hora de consulta; una lista vacía y una falla presentan mensajes distintos.	LP-10 / LP-12 / LP-14	02	Pendiente / fuera del sprint
HU-LP-09 Could · 3 pt	Como supervisor, quiero recuperar pedidos sin procesar, para evitar que queden olvidados.	Si la publicación falla, el pedido sigue consultable y un mecanismo de recuperación lo procesa sin crear otro id.	LP-11	01,03	Pendiente / fuera del sprint
HU-LP-10 Could · 1 pt	Como operador, quiero bloquear el botón mientras envío, para evitar solicitudes repetidas.	Mientras hay una creación pendiente, el botón no permite otro envío; ante respuesta perdida advierte consultar antes de repetir.	LP-16 / LP-17	01	Pendiente / fuera del sprint

Método MoSCoW: Must obligatorio, Should importante, Could ampliación. Escala: 1 simple, 2 varios casos, 3 integración o recuperación. Historias seleccionadas: HU-LP-01 a 04, total 7 puntos. Las cuatro quedaron verificadas en Docker; las seis restantes conservan estado pendiente o propuesto.

LOGISTPULSE plan de Sprint 1

SPR LP 01

Sprint Goal: Crear un pedido ficticio, consultarlo y demostrar su procesamiento automático hasta READY.

Duración propuesta: una semana, del 05 al 11 de octubre de 2026. Historias seleccionadas: HU-LP-01, HU-LP-02, HU-LP-03, HU-LP-04. 7 puntos en total.

Equipo y capacidad

Product Owner: Martín Endara. Scrum Master: Antonio Muñoz. Desarrollo: Martín, Antonio y Marco Bonilla. Roles propuestos para organizar el trabajo, sin afirmar contribuciones aún no registradas.

Capacidad propuesta para LOGISTPULSE: 18 horas de trabajo total. Referencia: tres integrantes con 6 horas disponibles cada uno. El equipo debe confirmar disponibilidad. Los puntos miden esfuerzo relativo; no son horas.

Historia	Tarea técnica principal	Responsable propuesto	Horas
HU-LP-01	Identificar contrato y probar creación en la API original.	Marco	2
HU-LP-02	Probar consulta, límite y orden de pedidos.	Marco	2
HU-LP-03	Probar que el pedido llegue al worker mediante evento.	Marco + Antonio	3
HU-LP-04	Capturar WAITING, PREPARING y READY del mismo id.	Antonio	3
Transversal	Backlog, diagrama MVC y matriz.	Martín	4
Transversal	README, PR, revisión cruzada y evidencia.	Martín + Antonio	3
Transversal	Ensayo de explicación del flujo.	Los tres	1

Definition of Done

Cada historia seleccionada cumple su criterio y registra prueba real. La base heredada y el aporte nuevo se distinguen. README, diagrama y matriz coinciden con el repositorio. La aceptación, la evidencia y la documentación están completas. Para cerrar la Definition of Done falta que Antonio o Marco revise o comente el PR #1, se atiendan observaciones y se fusione. No se incluyen secretos.

Prueba y evidencia

```
docker compose up -d --build --wait
python scripts/value-recovery-smoke.py --project LOGISTPULSE
```

Salida verificada: evidencias/sprint1.json. Resultado PASS, pedido ORD-656EE9; POST 201 con WAITING, GET 200 y el mismo id observado en WAITING, PREPARING y READY. Tras recrear contenedores sin borrar volúmenes, API y PostgreSQL conservaron el pedido en READY. Commit probado: 370b8dd90a637546400304ba63161fbbc0b85d4b.

LOGISTPULSE arquitectura MVC

LOGISTPULSE · MVC del flujo del Sprint 1

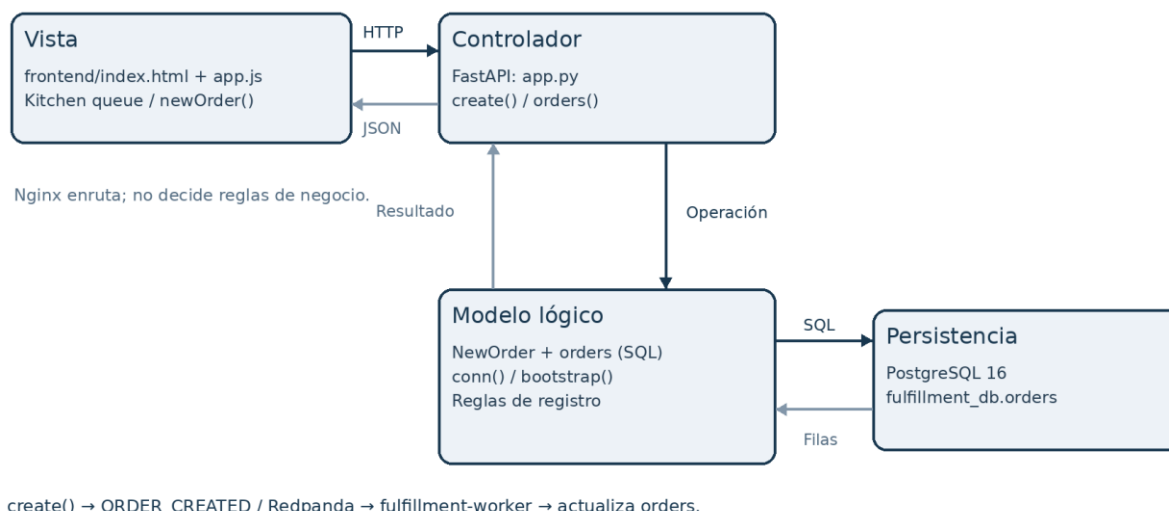


Figura 1. MVC-LP-01. Vista, Controlador, Modelo y respuesta del flujo seleccionado.

Parte	Elemento concreto y responsabilidad
Vista	frontend/index.html y app.js: Kitchen queue. fulfillment() muestra pedidos; newOrder() crea un pedido demo y vuelve a consultar.
Controlador	Rutas FastAPI de services/fulfillment-api/app.py: create(), orders() y health(). Reciben HTTP, validan tipos con Pydantic y devuelven JSON.
Modelo lógico	NewOrder representa la entrada; orders es la entidad persistida. conn()/bootstrap() y SQL realizan el acceso a datos. El worker aplica la preparación.
Persistencia y límites	PostgreSQL fulfillment_db.orders. create() publica ORDER_CREATED en Redpanda; fulfillment-worker actualiza PREPARING y READY. Nginx es el punto de entrada.

Justificación

FastAPI funciona como Controlador REST y la Vista HTML/JS consume sus rutas. La separación MVC es lógica: la plantilla concentra rutas, modelo de entrada y SQL en app.py; no afirmamos que existan módulos Model/Repository separados. La arquitectura diseñada identifica esas responsabilidades y permite extraer el acceso a datos más adelante. El worker es procesamiento de dominio, no una pantalla ni un Controlador HTTP. El patrón adicional es procesamiento asíncrono por eventos.

Límites conocidos

Guardar el pedido y publicar el evento no es una única transacción distribuida. Si la publicación falla, no hay recuperación garantizada. NewOrder no impide totales negativos. Estas mejoras permanecen fuera del Sprint 1. /health no comprueba por sí solo la base de datos.

LOGISTPULSE matriz de trazabilidad

Una fila por historia del Sprint 1. Origen INC-LP-01 y hallazgos del PDF inicial. El PR agrega documentación y pruebas; las clases de negocio permanecen identificadas como código heredado. T-LP-01 a T-LP-04 aprobaron con evidencia del mismo pedido.

Necesidad / hallazgo	HU / aceptación	Prioridad / puntos	Sprint Goal	Modelo / Vista / Controlador	Rama / PR / cambios	Servicio Docker	Prueba / resultado	Evidencia
INC-LP-01 LP-01 / LP-02	HU-LP-01 201 + id WAITING	Must 2 pt	SPR-LP-01 Pedido hasta READY	MVC-LP-01 NewOrder / orders SQL / worker; Kitchen queue; create() / orders() FastAPI	docs/value-recovery-lp PR #1 Cambios R1–R4	console + fulfillment- api + postgres + redpanda + fulfillment-worker	T-LP-01 PASS: 201 + WAITING	sprint1.json ORD-656EE9
INC-LP-01 LP-03	HU-LP-02 200, máximo 20, id presente	Must 1 pt	SPR-LP-01 Pedido hasta READY	MVC-LP-01 NewOrder / orders SQL / worker; Kitchen queue; create() / orders() FastAPI	docs/value-recovery-lp PR #1 Cambios R1–R4	console + fulfillment- api + postgres + redpanda + fulfillment-worker	T-LP-02 PASS: 200 + id presente	sprint1.json ORD-656EE9
INC-LP-01 LP-04	HU-LP-03 Mismo id PREPARING	Must 2 pt	SPR-LP-01 Pedido hasta READY	MVC-LP-01 NewOrder / orders SQL / worker; Kitchen queue; create() / orders() FastAPI	docs/value-recovery-lp PR #1 Cambios R1–R4	console + fulfillment- api + postgres + redpanda + fulfillment-worker	T-LP-03 PASS: mismo id PREPARING	sprint1.json ORD-656EE9
INC-LP-01 LP-05	HU-LP-04 WAITING → PREPARING → READY ≤60 s	Must 2 pt	SPR-LP-01 Pedido hasta READY	MVC-LP-01 NewOrder / orders SQL / worker; Kitchen queue; create() / orders() FastAPI	docs/value-recovery-lp PR #1 Cambios R1–R4	console + fulfillment- api + postgres + redpanda + fulfillment-worker	T-LP-04 PASS: WAITING → PREPARING → READY	sprint1.json + entorno commit 370b8dd

Referencias de cambios: R1 = README.md; R2 = docs/value-recovery/BACKLOG.md, MVC.md, MVC.svg y TRAZABILIDAD.csv; R3 = scripts/value-recovery-smoke.py; R4 = evidencias/*.json y documentos finales. Rama docs/value-recovery-lp; PR #1.

T-LP-01/02/03/04 se ejecutaron realmente en Docker y registraron PASS para ORD-656EE9. El commit probado es 370b8dd90a637546400304ba63161fbbc0b85d4b. La persistencia también se verificó después de docker compose down/up sin eliminar volúmenes.

Repositorio README y ciclo colaborativo

LOGISTPULSE

La carpeta conserva la aplicación original y agrega README de entrega, backlog, MVC, matriz, evidencia y test de dominio. Base pública revisada: randUSFQ/LOGISTPULSE-GOLDEN, commit 74fe4cdbedb32c55a7848b35195229adc58578b6. Ese repositorio es la fuente heredada, no el repositorio del equipo.

Fuente original de LOGISTPULSE

Repositorio del equipo: <https://github.com/martinendara-Dev/LOGISTPULSE-MartinEndara>. PR de HU-LP-01 a HU-LP-04: #1. Rama docs/value-recovery-lp. El README cubre propósito, stack, estructura, variables, comandos, puertos y pruebas. Commit probado: 370b8dd90a637546400304ba63161fbbc0b85d4b. El PR queda abierto hasta una revisión humana de Antonio o Marco; no se ha fusionado.

Qué debe demostrar la evidencia

Evidencia	Qué demuestra
Commits de la rama	Rama con commits claros y commit probado 370b8dd.
PR abierto y su URL	PR #1 abierto y vinculado a HU-LP-01 a HU-LP-04.
Comentario o revisión de otro integrante	Pendiente: revisión real de Antonio o Marco.
PR fusionado y commit	Pendiente hasta revisión y controles.
sprint1.json y capturas del stack	PASS real y persistencia verificada; sin secretos.

La evidencia versionada no contiene tokens ni .env. sprint1.json conserva respuestas y estados; verificacion-entorno.json registra herramientas, salud y persistencia. Los enlaces permiten comprobación externa. Si la consigna exige capturas, el equipo puede añadirlas sin sustituir estos datos reproducibles.

Guía sencilla para explicar tu parte

La secuencia que debes recordar

Necesidad → historia → prioridad y sprint → MVC → PR → servicio → prueba. Mantén el mismo ejemplo durante toda la explicación.

LOGISTPULSE en un minuto

«El operador necesita saber qué pasa con su pedido. El sprint crea uno y sigue su preparación. La Vista Kitchen queue llama a FastAPI; PostgreSQL guarda WAITING y Redpanda transporta ORDER_CREATED. El worker consume el evento y pasa el mismo id por PREPARING y READY. La prueba real siguió ORD-656EE9, por lo que no usamos pedidos distintos para aparentar la secuencia. La matriz une cada paso con su historia y evidencia».

Pregunta probable	Respuesta corta
¿Por qué este sprint?	Permite demostrar un pedido completo desde su creación hasta estar listo.
¿Qué diferencia backlog y sprint?	El backlog contiene todo lo propuesto; el sprint selecciona cuatro historias para lograr una meta.
¿Los puntos son horas?	No. Son esfuerzo relativo. La capacidad se controla aparte con horas disponibles.
¿MVC significa tres servidores?	No. Separa responsabilidades de Vista, Controlador y Modelo dentro del diseño.
¿El worker es un Controlador?	No es Controlador HTTP; procesa eventos y ejecuta la preparación del dominio.
¿Qué desarrollaron ustedes?	El laboratorio es la base. Nuestro PR agrega artefactos y pruebas; las mejoras futuras se identifican como pendientes.
¿Qué prueba que está terminado?	Criterios ejecutados y evidencia PASS sobre el commit probado. Para cerrar el DoD aún faltan revisión humana y merge.

Para ensayar: explica HU-LP-01 y HU-LP-04 siguiendo sus filas. Puedes afirmar que Docker pasó y que el PR #1 está abierto. No afirmes revisión ni fusión hasta que Antonio o Marco las realice.